Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»

FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»

Field of Study:
«Computer Science»

Тема	**Размещение задач с учётом дискового ввода-вывода в системах оркестрации контейнеров**
Topic	**Disk I/O-Aware Task Placement in Container Orchestration**
Работу выполнил / *Prepared by*	Галиев Арсен / Galiev Arsen
Руководитель выпускной квалификационной работы / *Final Qualification Work Supervisor*	Садовых Андрей / Sadovykh Andrey
Соруководитель / *Co-supervisor*	Браун Пьер-Филипп / Braun Pierre-Philipp

подпись / signature

подпись / signature

подпись / signature

Иннополис, Innopolis, 2027

Contents

Abstract **3**

1 Introduction **4**

1.1 Research Aim and Objectives 5

2 Literature Review **8**

2.1 The Task of Selecting a Node for Workload Placement 8

2.2 Cluster State Management for Task Placement Decisions 9

2.3 Metrics, Resources, and Node Selection Conditions 10

2.4 Disk Input/Output and Workload Interference 11

2.5 Approaches to Container Workload Placement Based on Resources and Disk I/O 12

2.6 Conclusions 14

Bibliography cited **16**

Abstract

Container orchestration systems select nodes for containerized workloads based on task requirements and available resources. For I/O-intensive workloads, disk activity may also be relevant to placement decisions. This study investigates how observed disk input/output information affects the placement of containerized workloads. It proposes an experimental comparison of placement with and without an additional disk I/O signal, focusing on disk load distribution and workload performance.

Chapter 1

Introduction

Containerization is a technology that isolates an application together with its dependencies [1]. Production applications increasingly consist of multiple containers that can be deployed across different computing nodes. Managing such applications manually becomes more complex because changes in the infrastructure require a response, and the containers and their number must be maintained.

Container orchestration systems, such as Kubernetes and Docker Swarm, help manage the tasks involved in maintaining a container cluster [2], [3]. One function of an orchestration system is **task scheduling**: selecting how tasks should be executed according to specified requirements and available resources. During scheduling, the scheduler makes a **task placement** decision by selecting the computing node on which a task will run [2], [4].

Applications use different computing node resources. Task placement in a cluster can account for the required CPU load and memory consumption [5]. However, disk input/output (I/O) performance can also be important for applications with tasks that frequently read and write data. Such applications include databases, caching systems, and other services that store and process data. There-

fore, disk I/O information may be useful when selecting a node for such workloads in a container cluster.

Disk load can vary across cluster nodes with different physical disks. If a new disk I/O-intensive task is placed on a node whose disk subsystem is already under I/O load, the task may compete with other tasks for disk access [6]. Meanwhile, this resource may be more readily available on another node.

This observation raises the question of whether information about *actual disk I/O* can be used in task placement and how its use may affect load distribution and application performance. When conducting research, it is necessary to consider the chosen metric, workload characteristics, storage architecture, and the relevance of the measurements at the time a decision is made.

The research problem is therefore to assess how the use of disk I/O information in node selection affects the distribution and performance of I/O-intensive containerized tasks under specified experimental conditions.

1.1 Research Aim and Objectives

The **aim of this study** is to investigate the effect of actual disk I/O information on the placement of containerized workloads.

The study has the following objectives:

1. examine container orchestration, cluster state management, and task placement to the extent required for the study;

2. review existing resource-aware workload placement approaches, with particular attention to disk I/O;

3. select an orchestration system and design an experimental testbed for running I/O-intensive containerized workloads;

4. investigate the feasibility of, and implement or configure, a mechanism that supplies a selected disk I/O metric for placement decisions;

5. compare workload placement with and without an additional I/O signal under specified conditions;

6. analyze the results, experimental limitations, and factors affecting the validity of the conclusions.

The primary research question is as follows:

How does the use of actual disk I/O information affect the placement of I/O-intensive containerized workloads compared with placement without an additional I/O signal?

The study also addresses the following subquestions:

- Which disk I/O metrics can be collected and used for placement decisions in the selected environment?
- How does the use of an I/O signal affect the distribution of disk load across nodes?
- How does it affect workload performance metrics?
- What limitations arise from the selected orchestration system, measurement method, and testbed configuration?

The study does not assume that accounting for disk I/O will necessarily improve placement or performance. This effect will be evaluated experimentally.

The proposed testbed will use multiple computing nodes whose disk loads can be assessed separately. Its specific configuration will be determined during the experimental design phase.

Chapter 2

Literature Review

2.1 The Task of Selecting a Node for Workload Placement

Container workload placement consists of selecting a computing node to execute a task. The scheduler considers the workload requirements and the characteristics of available nodes when making this decision. The specific selection rules depend on the orchestration system and its configuration [4].

In Kubernetes, the scheduler selects a node for a Pod object. First, it excludes nodes that do not satisfy the Pod requirements. The scheduler then scores the remaining nodes according to the applicable rules and selects one of them. If no suitable node is available, the Pod remains unscheduled until a suitable node becomes available [4].

Node selection criteria may use information about resource status in addition to task requirements. Li et al. proposed Kubernetes algorithms that consider the disk I/O load of nodes when selecting a location for a new Pod [7]. The methods

used to obtain this information and the experimental results are discussed later in the review.

Thus, placement connects workload requirements, the state of available nodes, and the scheduler selection rules. Further consideration of these rules makes it possible to determine how information about disk input/output can contribute to placement decisions.

2.2 Cluster State Management for Task Placement Decisions

The scheduler uses information about the cluster state when selecting a node. If multiple nodes participate in management, the system must also ensure that the management state is consistent across them. The mechanism for maintaining this consistency depends on the orchestrator architecture [3], [4].

In Docker Swarm, nodes perform manager and worker roles. Manager nodes manage the cluster and maintain its consistent state using the Raft algorithm [3], [8]. Confirmation of state changes requires a majority of manager nodes, known as a quorum [8]. The scheduler operating in the Swarm management layer selects a node to execute a task according to the configured strategy and the orchestrator version [3]. Worker nodes, in turn, exclusively execute the container tasks assigned to them. A manager node can also execute tasks if configured accordingly [3].

2.3 Metrics, Resources, and Node Selection Conditions

Resource requirements, such as RAM and CPU consumption, can be specified when a task is created. The scheduler considers these requirements when determining eligible nodes according to the rules of the orchestration system [4].

Declared resource requirements help determine whether a task can be placed on a node. Actual consumption, meanwhile, shows how an already running workload uses resources. The scheduler can use the specified task requirements and information about node status to make a decision, depending on the capabilities and configuration of the system [4], [7].

In addition to resource requirements, placement constraints and preferences can influence node selection. In Docker Swarm, a constraint specifies a mandatory condition that a node must satisfy. A preference, by contrast, specifies a desired distribution of tasks but does not guarantee that this distribution will be achieved [9].

The scheduler can consider several criteria simultaneously. Menouer proposes the KCSS strategy for Kubernetes, which evaluates nodes using several characteristics, including CPU, memory, and disk space usage [5]. The author uses the multicriteria TOPSIS method to rank suitable nodes [5].

In the study by Menouer [5], the disk usage metric is defined as the ratio of occupied disk space to total capacity [5]. This metric describes **disk capacity**, rather than read or write speed or operation latency directly. Therefore, an examination of I/O-intensive workloads requires a separate study of disk input/output metrics and methods for measuring them.

2.4 Disk Input/Output and Workload Interference

Disk input/output includes data read and write operations. The nature of a workload depends, among other factors, on the ratio of read to write operations, their size, and the data access pattern: sequential or random. These characteristics must be considered when comparing I/O workloads: the same number of operations can correspond to different volumes of transferred data.

Several metrics are used to assess disk subsystem performance.

IOPS describes the number of input/output operations per unit of time, whereas **throughput** describes the volume of data read or written per unit of time. For example, the iostat utility separately reports the number of completed read and write requests per second and the volume of transferred data [10].

Another metric is **I/O latency**, or the time required to complete an operation. Its interpretation requires consideration of the level at which it is measured. For example, the await metric in iostat reflects the average service time of requests issued to a device, including time spent waiting in the queue. The utility also provides a metric for the average queue length of such requests [10]. Observed values depend on the measurement level. Application metrics reflect the result of application execution, whereas device statistics concern requests that have reached the device.

Workloads can affect one another's performance when they share hardware resources. Xavier et al. [11] studied the performance isolation of disk-intensive applications in LXC containers. The authors compared a workload running alone with the same workload running concurrently with another workload on the same hardware. In the configurations studied, the effect depended on the combination of workloads: the authors observed both performance degradation and a scenario

without noticeable interference [11].

Thus, placing an I/O-intensive task requires consideration of both available storage capacity and how the node disk subsystem is used. The choice of metric depends on the workload under consideration and the measurement level. The following section examines studies that use information about resources and disk input/output directly when selecting a node for a container workload.

2.5 Approaches to Container Workload Placement Based on Resources and Disk I/O

Container workloads can affect one another's performance when they share the resources of a single node. To reduce this effect and use resources more efficiently, researchers propose considering workload characteristics and node status when making placement decisions [12], [13].

Mao et al. [12] proposed the **DRAPS** scheme for placing Docker containers in a heterogeneous cluster. The authors assume that cluster nodes may differ in available resources and that container services may differ in their resource consumption patterns. In the proposed system, worker nodes collect information about container resource usage, including CPU time, memory, network I/O, and block I/O. Based on these data, DRAPS makes task placement decisions that account for available node resources [12].

A distinguishing feature of DRAPS is its consideration of several resource types within a single placement scheme. This approach allows differences between workloads and nodes to be considered. However, the inclusion of block I/O among the collected metrics does not, by itself, demonstrate improved performance of

disk-intensive applications. The article results should be interpreted in relation to the workloads and testbed configuration examined by the authors [12].

Chiang [13] proposed another placement approach that considers **resource contention and interference between container workloads**. The author studied distributed applications running concurrently with other workloads and proposed a scheduler prototype that uses clustering algorithms to make placement decisions [13].

The experiments by Chiang used a Docker Swarm cluster on Amazon EC2 virtual machines. The applications studied included the Cassandra distributed storage system and other workloads with different resource requirements. The results show that workload co-location can affect application performance and that the results depend on the nature and intensity of the competing workload [13]. For the present study, the article is useful as an example of accounting for interference when selecting a node, although it considers resource contention more broadly than disk I/O alone.

Li et al. [7] directly consider disk input/output when placing Pods in Kubernetes. The authors proposed two node scoring algorithms: **Balanced-Disk-IO-Priority (BDI)** and **Balanced-CPU-Disk-IO-Priority (BCDI)**. The BDI algorithm aims to balance disk I/O load across cluster nodes. BCDI additionally considers the relationship between CPU and disk I/O load on an individual node [7].

The authors use a Prometheus-based monitoring system to obtain information about the cluster state. The proposed algorithms are integrated with the Kubernetes scheduler through the scheduler extension mechanism and contribute to scoring nodes for a new Pod [7].

The experimental cluster of Li et al. [7] consisted of four OpenStack virtual

machines: one control node and three worker nodes. The study used Kubernetes version 1.11.6. The authors compared the proposed algorithms with standard scheduling algorithms under their testbed conditions and obtained results indicating improvements in selected metrics in the scenarios studied [7]. However, the results depend on the virtual machine configuration, workload characteristics, and disk resources used; therefore, they should not be transferred to other environments without further verification.

The studies reviewed use different bases for decision-making. DRAPS considers multiple resource types and cluster heterogeneity [12], Chiang studies placement that accounts for contention between workloads [13], and Li et al. directly include disk I/O in Kubernetes node scoring [7]. These studies show that researchers have already investigated the use of disk I/O information and workload interference. Therefore, the present study examines the effect of using such information on container workload placement and performance **under the conditions of the selected experimental testbed**, rather than the idea of using this information itself.

2.6 Conclusions

Container workload placement consists of selecting a node to execute a task. The scheduler can consider task requirements, node characteristics, and specified placement conditions when making a decision. The specific set of criteria depends on the orchestration system and the scheduling mechanism used [4], [9].

The studies reviewed show that resources can be evaluated using several criteria simultaneously. Available storage capacity must be distinguished from disk input/output characteristics: the volume of occupied space does not directly

describe the number of operations, data transfer rate, or I/O latency [5], [10].

Container workloads running concurrently on the same node can interfere with one another. The nature of this effect depends on the combination of workloads and their execution conditions [11], [13]. Researchers have proposed placement approaches that account for node heterogeneity, resource consumption, and contention between workloads [12], [13].

Li et al. demonstrated one way to use disk I/O when scoring Kubernetes nodes [7]. Thus, researchers have already considered disk input/output information in container placement. However, individual studies obtained their results under specific conditions, and those results may not necessarily be reproduced with different workload and infrastructure characteristics.

Based on the review, the present study aims to investigate how the use of information about actual disk I/O when selecting a node affects the placement and performance of I/O-intensive container workloads. This investigation requires a method for obtaining and passing the metric to the scheduler, followed by a comparison of placement with and without this information. The orchestration system, metrics, and testbed configuration must be determined when designing the experiment.

Bibliography cited

[1] Docker. “What is a container?” [Online]. Available: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/ (accessed Nov. 1, 2026).

[2] C. Carrión, “Kubernetes scheduling: Taxonomy, ongoing issues and challenges,” *ACM Computing Surveys*, vol. 55, no. 7, 2023. DOI: 10.1145/3539606.

[3] Docker. “How nodes work,” [Online]. Available: https://docs.docker.com/engine/swarm/how-swarm-mode-works/nodes/ (accessed Nov. 1, 2026).

[4] Kubernetes. “Kubernetes scheduler,” [Online]. Available: https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/ (accessed Nov. 1, 2026).

[5] T. Menouer, “KCSS: Kubernetes container scheduling strategy,” *The Journal of Supercomputing*, vol. 77, pp. 4267–4293, 2021. DOI: 10.1007/s11227-020-03427-3.

[6] V. Medel, U. Arronategui, O. Rana, J. Á. Bañares, and R. Tolosana-Calasanz, “Modeling and characterizing service interference in dynamic infrastructures,” *IEEE Access*, vol. 11, pp. 21 387–21 403, 2023. DOI: 10.1109/ACCESS.2023.3250606.

[7] D. Li, Y. Wei, and B. Zeng, "A dynamic I/O sensing scheduling scheme in Kubernetes," in *Proceedings of the 2020 4th International Conference on High Performance Compilation, Computing and Communications*, 2020, pp. 14–19. DOI: 10.1145/3407947.3407950.

[8] Docker. "Raft consensus in swarm mode," [Online]. Available: https://docs.docker.com/engine/swarm/raft/ (accessed Nov. 1, 2026).

[9] Docker. "Deploy services to a swarm: Placement constraints and preferences," [Online]. Available: https://docs.docker.com/engine/swarm/services/#placement-constraints (accessed Nov. 1, 2026).

[10] S. Godard. "iostat(1) — linux manual page," [Online]. Available: https://man7.org/linux/man-pages/man1/iostat.1.html (accessed Nov. 1, 2026).

[11] M. G. Xavier, I. C. de Oliveira, F. D. Rossi, R. D. dos Passos, K. J. Matteussi, and C. A. F. de Rose, "A performance isolation analysis of disk-intensive workloads on container-based clouds," in *Proceedings of the 2015 23rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*, 2015, pp. 253–260. DOI: 10.1109/PDP.2015.67.

[12] Y. Mao, J. Oak, A. Pompili, D. Beer, T. Han, and P. Hu, "DRAPS: Dynamic and resource-aware placement scheme for Docker containers in a heterogeneous cluster," in *Proceedings of the 2017 IEEE 36th International Performance Computing and Communications Conference (IPCCC)*, 2017, pp. 1–8. DOI: 10.1109/PCCC.2017.8280474.

[13] R. C. Chiang, "Contention-aware container placement strategy for Docker Swarm with machine learning based clustering algorithms," *Cluster Computing*, vol. 26, no. 1, pp. 13–23, 2023. DOI: 10.1007/s10586-020-03210-2.